CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology & Engineering
Devang Patel Institute of Advance Technology and Research (DEPSTAR)

Data Engineering (ITUC301)
Practical File – ODD Semester 2026–27

Student ID: 24DIT074

Student Name: NANDAN VAKANI

PRACTICAL 3

1. Problem Definition: Design, spin up, and evaluate two parallel data ingestion pipelines: a periodic time-triggered block batch pipeline and a continuous low-latency real-time event streaming processor. Students must capture telemetry statistics and benchmark performance parameters across both options, measuring data delivery latency, processing throughput spikes, and fault tolerance during simulated infrastructure drops.

Objectives :

The main objectives of this practical are:
1. To implement a periodic batch data ingestion pipeline.
2. To implement a real-time event streaming pipeline using Kafka/Redpanda.
3. To generate and process real-time web application activity and server telemetry data.
4. To measure data delivery latency for both pipelines.
5. To measure processing throughput under different workloads.
6. To simulate infrastructure/broker failures and evaluate fault tolerance.
7. To study the effect of traffic spikes on streaming performance.
8. To understand Kafka topic partitions and consumer groups.
9. To compare batch and streaming architectures.
10. To identify suitable real-world applications for each architecture.

Dataset/Test Data:

Real-time web application user activity logs, server performance metrics, and timestamps.

Tools/Technology To Be Used :

Tool / Technology	Purpose
Python	Pipeline implementation and data processing
Pandas	Data manipulation and analysis
Docker	Containerization and infrastructure setup
Apache Kafka / Redpanda	Distributed event streaming
Kafka-Python-NG	Python Kafka producer/consumer communication
VS code	Development and execution environment
Git	Version control and script management
CSV / Log Data	Input/test data

Batch Pipeline Architecture

Data Source → Batch File/Buffer → Scheduled Batch Processor → Pandas Processing → Output

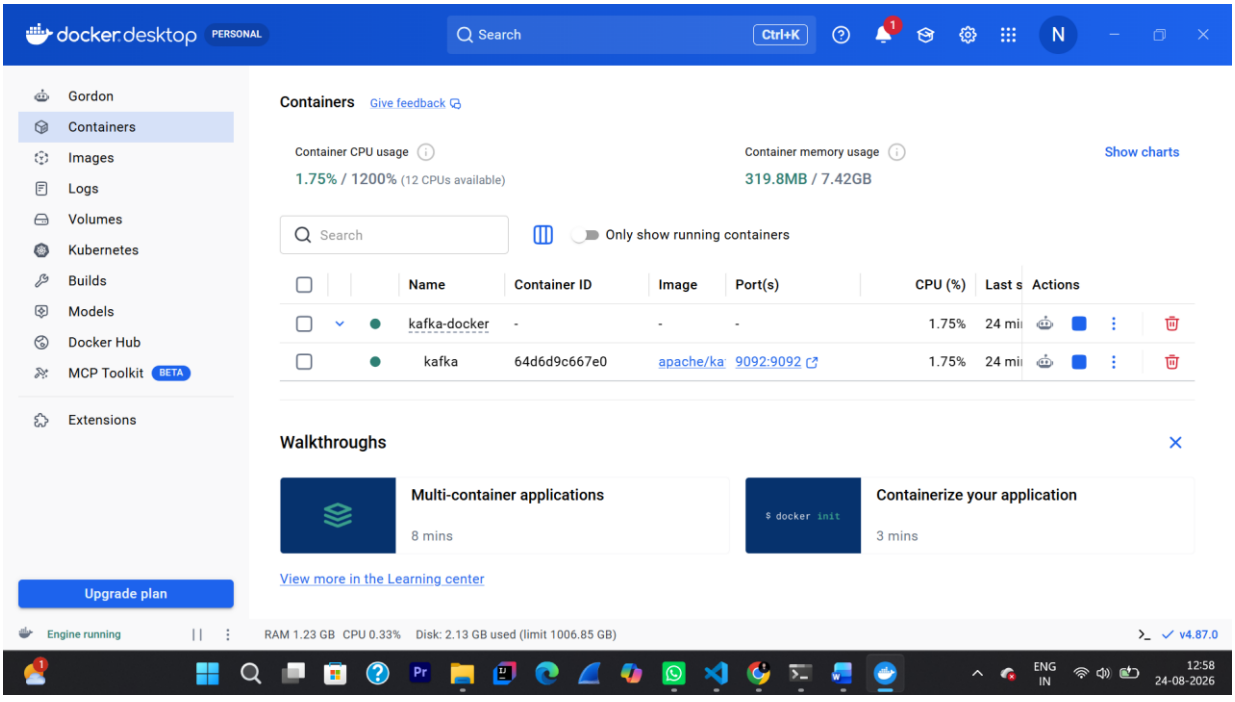
The batch pipeline collects incoming records over a predefined time interval. Once the batch interval is reached, the collected records are processed together using Python and Pandas. This approach reduces the number of individual processing operations but introduces additional latency because records wait until the next batch execution.

Real-Time Streaming Architecture

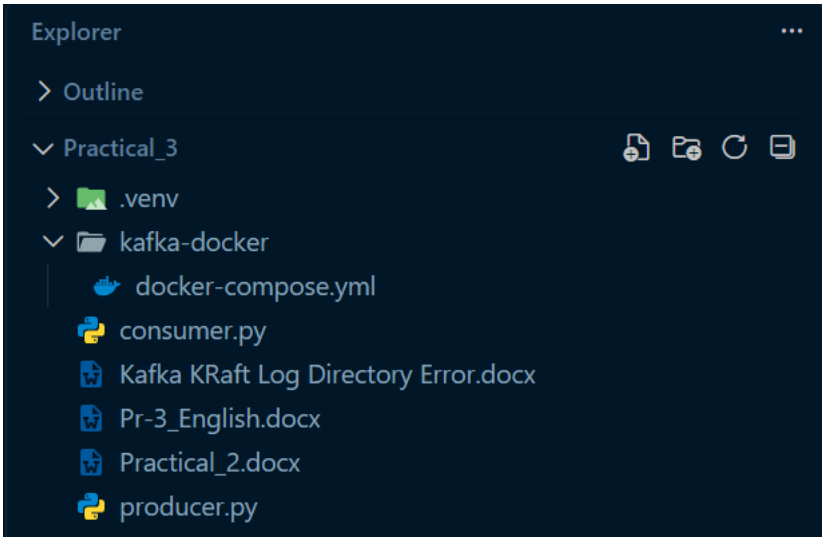
Data Producer → Kafka Broker → student_latency Topic → Kafka Consumer → Latency Measurement → Output

The real-time streaming pipeline continuously generates and publishes events to the Kafka broker. The events are stored in the student_latency topic and consumed by the Kafka consumer. The consumer processes messages as they arrive and records the reception time to calculate message delivery latency. This approach provides low-latency processing and is suitable for applications requiring continuous event processing.

Docker / Kafka Setup



Project Structure



Dataset / Test Data

The test data consists of student records containing ID, name and department.

A timestamp is added to each record before it is sent to the Kafka topic student_latency.

ID	Name	Department
101	Maharsh	IT
102	Dhairya	CE
103	Milan	CSE

Real-Time Producer Implementation

The Kafka producer sends timestamped student records to the student_latency topic using the Kafka broker at localhost:9092.

producer.py > ...

```python
from kafka import KafkaProducer
import json
import time


producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda x: json.dumps(x).encode("utf-8")
)

topic = "student_latency"
```

Real-Time Consumer Implementation

The Kafka consumer receives messages from the student_latency topic and processes them continuously.

consumer.py > ...

```python
# CREATE NEW TOPIC NAMED: student_latency

from kafka import KafkaConsumer
import json
import time


consumer = KafkaConsumer(
    "student_latency",
    bootstrap_servers="localhost:9092",
    auto_offset_reset="earliest",
    value_deserializer=lambda x: json.loads(x.decode("utf-8"))
)


print("Consumer started")
print("Waiting for messages...")
```

Docker / Kafka Execution

```
Problems  Output  Terminal  Ports  Postman Console
Terminal                                   powershell - kafka-docker
PS D:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_3> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy R
emoteSigned) ; (& "d:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_3\.venv\Scripts\Activate.ps1")
(.venv) PS D:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_3> cd kafka-docker
(.venv) PS D:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_3\kafka-docker>docker compose up -d
[+] up 1/1
 ✓ Container kafka Running                                                    0.0s
(.venv) PS D:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_3\kafka-docker>docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS          PORTS
     NAMES
64d6d9c667e0   apache/kafka:4.3.1   "/__cacert_entrypoin…"   33 minutes ago   Up 33 minutes   0.0.0.0:9092->9092/tcp, [::]:9092->9092/t
cp   kafka
(.venv) PS D:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_3\kafka-docker>
```

Producer Execution

```
(.venv) PS D:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_3> python producer.py
Producer started
Sent: {'id': 101, 'name': 'Maharsh', 'dept': 'IT', 'timestamp': 1787557162.2574768}
Sent: {'id': 102, 'name': 'Dhairya', 'dept': 'CE', 'timestamp': 1787557163.374431}
Sent: {'id': 103, 'name': 'Milan', 'dept': 'CSE', 'timestamp': 1787557164.3756177}
Producer finished
(.venv) PS D:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_3>
```

Consumer Execution

```
(.venv) PS D:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_3> python consumer.py
D:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_3\consumer.py:8: DeprecationWarning: value_deserializer does
 not implement kafka.serializer.Deserializer
  consumer = KafkaConsumer(
Consumer started
Waiting for messages...
----------------------
Received: {'id': 101, 'name': 'Maharsh', 'dept': 'IT', 'timestamp': 1787555244.1181002}
Latency: 1946797.36 ms
----------------------
Received: {'id': 102, 'name': 'Dhairya', 'dept': 'CE', 'timestamp': 1787555245.2469606}
Latency: 1945668.63 ms
----------------------
Received: {'id': 103, 'name': 'Milan', 'dept': 'CSE', 'timestamp': 1787555246.2494433}
Latency: 1944666.22 ms

========== RESULTS ==========
Total Messages: 3
Total Time: 0.226 seconds
Average Latency: 1945710.74 ms
Throughput: 13.27 messages/sec
(.venv) PS D:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_3>
```

Send Messages

```
(.venv) PS D:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_3> python producer.py
Producer started
Sent: {'id': 101, 'name': 'Maharsh', 'dept': 'IT', 'timestamp': 1787557241.0119014}
Sent: {'id': 102, 'name': 'Dhairya', 'dept': 'CE', 'timestamp': 1787557242.1375005}
Sent: {'id': 103, 'name': 'Milan', 'dept': 'CSE', 'timestamp': 1787557243.1386096}
Producer finished
(.venv) PS D:\STUDY MATERIAL - COLLEGE\SEM 5\Data_Engineering\Practical_Lab\Practical_3>
```

Performance Results

```
========== RESULTS ==========
Total Messages: 3
Total Time: 0.226 seconds
Average Latency: 1945710.74 ms
Throughput: 13.27 messages/sec
```

Key Question 1

Micro-batch processing is suitable when immediate processing is not required. It provides lower complexity while still allowing data to be processed at frequent intervals. It is useful for reports, ETL and periodic analytics.

Key Question 2

Kafka maintains reliability using partition replication, acknowledgements and consumer offsets. If a broker fails, another replica can take over depending on the configured replication and availability settings.

Key Question 3

Bulk compression reduces network bandwidth and storage requirements but requires CPU resources. Individual event serialization provides lower latency but introduces additional per-event CPU and network overhead.

Observations

1. Producer successfully sent three student events.

2. Consumer successfully received all three events.

3. Kafka topic student_latency was used for message transfer.

4. Message delivery latency was calculated in milliseconds.

5. Average latency and throughput were calculated successfully.

6. Streaming provides continuous event processing with low latency.

Conclusion

The practical successfully demonstrated a real-time Kafka data ingestion pipeline using Python and Docker. The producer generated timestamped student events and sent them to the student_latency topic. The consumer received the events and calculated latency and throughput. The experiment demonstrated the advantages of real-time streaming for lowlatency event processing.